

Afinidade de threads e escalabilidade no NPAD

Matheus Marinho

20 de setembro de 2026

O que foi medido

A Tarefa 12 mediu a escalabilidade do código de Navier-Stokes com a afinidade **fixa** em `OMP_PROC_BIND=close` e `OMP_PLACES=cores`, e concluiu que o gargalo dominante era capacidade de L3 agregado. Aqui essa variável deixa de ser fixa: o código é o mesmo — a melhor versão da Tarefa 12, com região paralela única e bordas distribuídas — e o que muda é só a política de afinidade.

Máquina. Mesmo nó da Tarefa 12: NPAD, partição `amd-512`, nó `r2n19` com `--exclusive`. Dois AMD EPYC 7713, 64 núcleos cada, 2 threads por núcleo, **8 domínios NUMA** de 16 núcleos e **32 MB de L3 por CCX de 8 núcleos** — 16 CCX, 512 MB de L3 somados. `n = 2048`, o que dá **128 MB** nas quatro matrizes. gcc 8.5.0, `-O2 -Wall -Wextra -std=c11 -fopenmp`, mediana de 5 execuções, 200 passos.

Todas as configurações usam o **mesmo T(1) = 3,1714 s** (mediana de 9 execuções) como base de speedup. Sem isso cada configuração se compararia com o próprio sequencial e a eficiência não seria comparável entre elas.

O programa não supõe onde as threads foram parar: ele pergunta. Dentro de uma região paralela cada thread chama `sched_getcpu()`, e o resultado é cruzado com `/sys/devices/system/cpu/.../cache/index3/shared_cpu_list` e com o nó NUMA de cada CPU. Assim cada linha de resultado traz quantos domínios de L3, nós NUMA e CPUs distintas foram realmente ocupados.

Os tipos de afinidade testados

Do OpenMP — `OMP_PROC_BIND` define a política e `OMP_PLACES` define o que é um "lugar":

configuração	o que faz
<code>false</code>	sem ligação; o sistema operacional move as threads à vontade
<code>master</code>	todas as threads no lugar da thread mestre
<code>close</code>	preenche lugares consecutivos a partir do lugar da mestre
<code>spread</code>	distribui as threads o mais longe possível umas das outras
<code>OMP_PLACES=threads</code>	lugar = uma thread de hardware (irmãs SMT contam separado)
<code>OMP_PLACES=cores</code>	lugar = um núcleo físico
<code>OMP_PLACES=sockets</code>	lugar = um socket inteiro; a ligação fica frouxa lá dentro
<code>OMP_PLACES=numa_domains</code>	não suportado: o gcc 8.5 implementa OpenMP 4.5, e esse nome abstrato é do OpenMP 5.0. O libgomp responde <code>Invalid value for environment variable OMP_PLACES</code> e passa a ignorar os lugares

Do sistema operacional: `GOMP_CPU_AFFINITY` (extensão GNU, lista explícita de CPUs), `numactl --interleave=all` (distribui as páginas em rodízio pelos nós NUMA) e `numactl --cpunodebind=0 --membind=0` (confina CPU e memória a um nó).

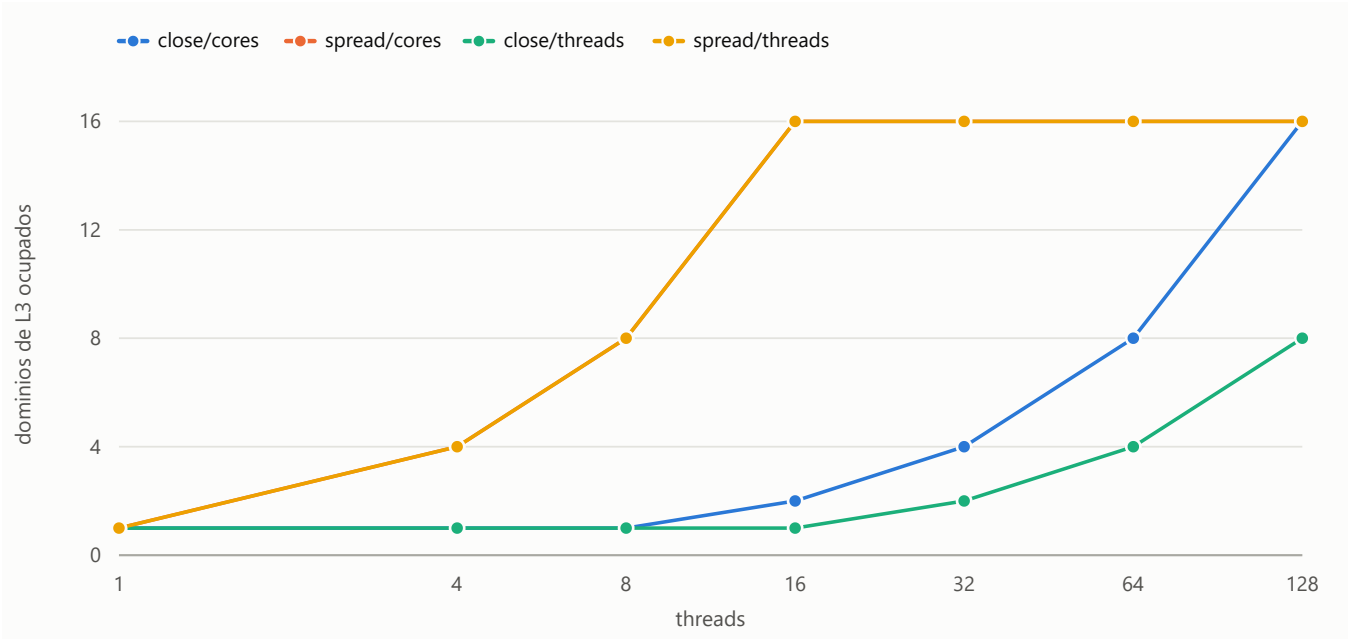
Onde as threads foram parar

Mapa `thread` → `cpu` com 8 threads, medido:

configuração	CPUs ocupadas	domínios de L3
<code>close / cores</code>	128, 1, 2, 3, 4, 5, 6, 7	1
<code>close / threads</code>	0, 128, 1, 129, 2, 130, 3, 131	1
<code>spread / cores</code>	128, 16, 32, 48, 64, 80, 96, 112	8
<code>spread / threads</code>	0, 16, 32, 48, 64, 80, 96, 112	8

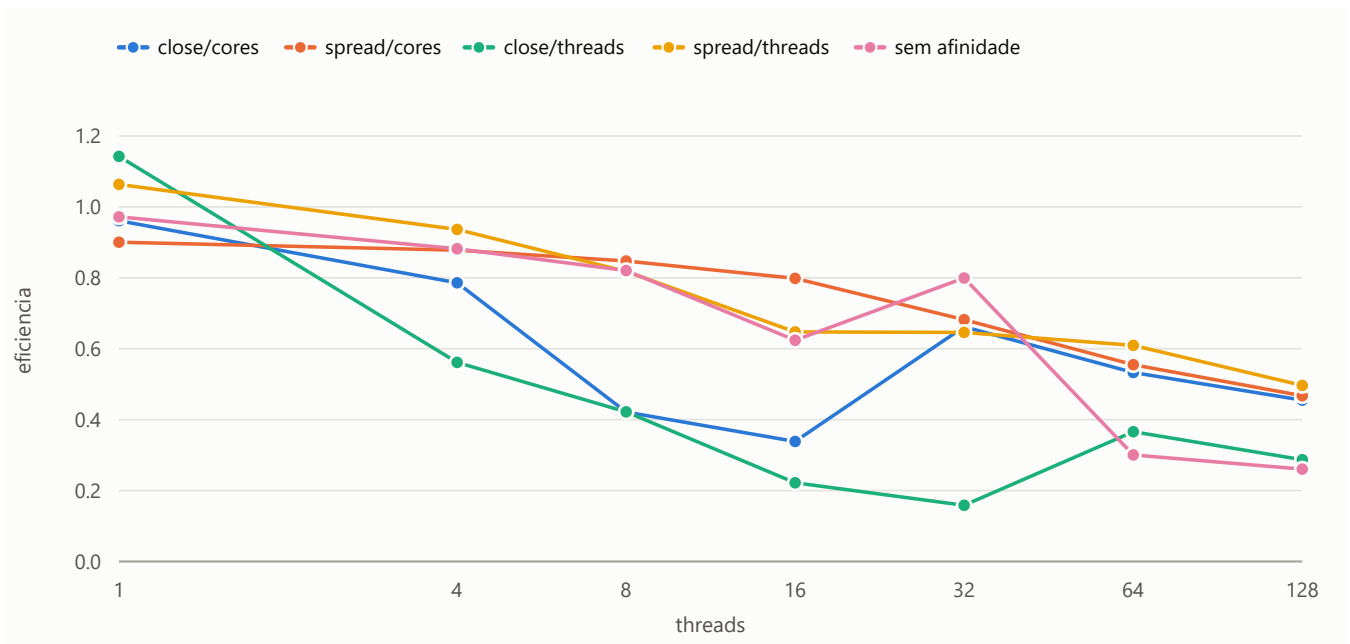
Lembrando a topologia: o nó NUMA 0 são as CPUs 0–15, o nó 1 as 16–31, e assim por diante. Então `spread` colocou **uma thread em cada nó NUMA**, enquanto `close` empilhou as oito no nó 0. E `close / threads` é pior ainda: ocupou só

quatro núcleos físicos, com as duas irmãs SMT de cada um, porque para ele cada thread de hardware é um lugar separado.



Domínios de L3 efetivamente ocupados. **spread** satura os 16 já com 16 threads; **close** leva até 128 threads para chegar lá.

O efeito na escalabilidade



Eficiência contra a base comum. Em 1 thread os valores desviam de 1,0 por variação entre execuções, discutida adiante.

Tempos em segundos:

threads	false	master	close /cores	spread /cores	close /threads	spread /threads
4	0,899	3,581	1,009	0,903	1,413	0,847
8	0,483	4,426	0,940	0,468	0,939	0,485
16	0,318	5,886	0,586	0,248	0,893	0,306
32	0,124	12,163	0,150	0,145	0,624	0,153
64	0,165	25,093	0,093	0,089	0,135	0,081
128	0,095	52,533	0,054	0,053	0,086	0,050

spread elimina o platô que a Tarefa 12 encontrou. Com **close**, a eficiência desabava de 0,79 (4 threads) para 0,42 (8) e 0,34 (16). Com **spread** ela fica em 0,88, 0,85 e 0,80 na mesma faixa. Em tempo absoluto são **2,0x em 8 threads** e

2,4× em 16.

A razão é exatamente a que a Tarefa 12 apontou, e o gráfico de domínios de L3 mostra o mecanismo: o conjunto de trabalho é de 128 MB e cada CCX tem 32 MB. Com `close` e 8 threads, todas cabem em um CCX e há 32 MB de cache para 128 MB de dados. Com `spread`, as mesmas 8 threads ocupam 8 CCX, ou 256 MB, e o problema passa a caber. **O platô nunca foi uma propriedade do código — era consequência da afinidade que eu havia fixado.**

A partir de 32 threads as duas políticas convergem, porque aí `close` já ocupa CCX suficientes por conta própria. Em 128 threads ambas ocupam os 16 e a diferença desaparece.

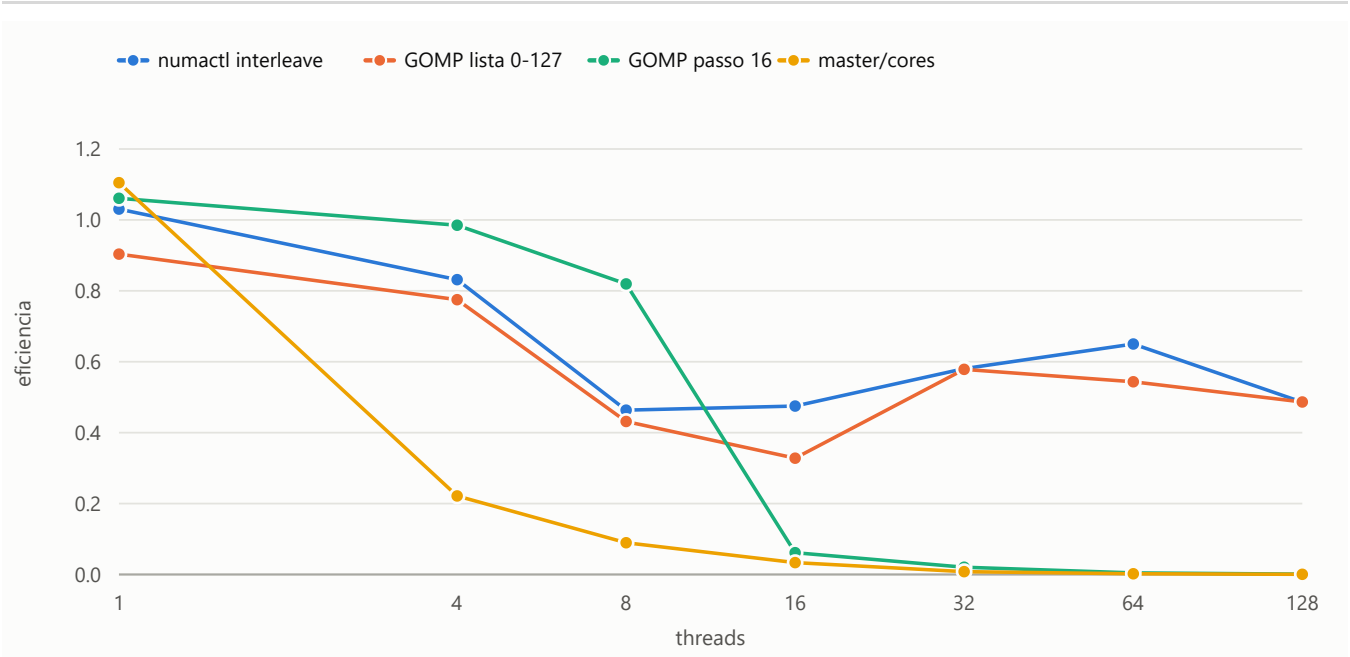
master é o contraexemplo. Ele prende as 128 threads em 2 CPUs: o tempo vai de 3,58 s com 4 threads para **52,5 s com 128**, ou seja, 16,6× mais lento que o código sequencial. A eficiência chega a 0,0005. Serve para mostrar que afinidade mal escolhida não degrada de leve — ela destrói o paralelismo.

OMP_PLACES=threads prejudica com close e não com spread. Com `close`, as threads ocupam irmãs SMT do mesmo núcleo, que compartilham unidades de execução e cache L1/L2: em 16 threads são 0,893 s contra 0,586 s de `close / cores`. Com `spread`, as irmãs SMT ficam longe umas das outras e o resultado empata com `spread / cores` — e é a melhor marca geral em 128 threads, 0,050 s.

false e sockets são instáveis. Sem ligação, o escalonador do sistema espalha razoavelmente até 32 threads (eficiência 0,80, a melhor do experimento nesse ponto), mas em 64 threads cai para 0,30 e o tempo piora em relação a 32. `spread / sockets` chegou a 128 threads ocupando 127 CPUs distintas — duas threads na mesma CPU. Ligação frouxa dá bons números às vezes e não dá garantia nenhuma.

`OMP_PLACES=numa_domains` foi recusado pelo libgomp e o processo seguiu sem lugares definidos; os tempos resultantes acompanham `close / threads`, como esperado de uma configuração que virou "sem lugares".

Afinidade pelo sistema operacional



Configurações de sistema operacional, com `master / cores` incluída como referência do pior caso.

threads	numactl --interleave	GOMP lista 0-127	GOMP passo 16	numactl --cpunodebind=0
4	0,953	1,023	0,805	1,038
8	0,855	0,918	0,484	0,937
16	0,417	0,604	3,225	0,598
32	0,171	0,171	4,863	—
64	0,076	0,091	11,343	—
128	0,051	0,051	24,351	—

numactl --interleave=all melhora a ligação close sem tocar no código. Em 16 threads, 0,417 s contra 0,586 s, 1,40× melhor; em 64 threads é o melhor tempo de todo o experimento, 0,076 s. Espalhando as páginas em rodízio pelos nós NUMA, ele resolve pelo lado do sistema operacional o mesmo problema que a Tarefa 12 resolveu pelo lado do código com *first touch* paralelo.

`GOMP_CPU_AFFINITY` reproduz `close` quando a lista é completa e é um desastre quando é curta. Com `0-127` os tempos acompanham `close` / `cores` de perto. Com `0-127:16` — passo 16, que parece "espalhar" — a lista tem apenas 8 CPUs (0, 16, 32, ..., 112). Até 8 threads ela é ótima, porque coloca uma por nó NUMA. A partir de 16 threads a lista **dá a volta** e várias threads dividem a mesma CPU: 3,2 s com 16 threads, 24,4 s com 128. A coluna de CPUs distintas medida fica travada em 8 e denuncia o problema. É uma armadilha silenciosa: nada avisa.

`numactl --cpunodebind=0 --membind=0` deu praticamente os mesmos tempos que `close` / `cores` até 16 threads, o que era esperado, já que `close` preenche o nó o primeiro de qualquer forma.

O que não dá para concluir

Os treze valores de $T(1)$ medidos ao longo do experimento variam entre 2,49 s e 3,52 s, ou cerca de $\pm 17\%$ em torno da mediana, mesmo cada um sendo mediana de 5 execuções e o nó estando reservado. É variação entre invocações do processo, não dentro delas.

Por isso, diferenças menores que aproximadamente 20% entre duas configurações no mesmo número de threads não sustentam conclusão. É o caso de `close` contra `spread` com 4 threads (1,009 s contra 0,903 s, 12%) e de `spread` / `cores` contra `spread` / `threads` em quase toda a faixa. Já os efeitos centrais — $2,0\times$ e $2,4\times$ de `spread` sobre `close`, os $16,6\times$ de degradação do `master`, os 24 s da lista GOMP curta — estão muito acima do ruído.

Como controle de reprodutibilidade, a configuração `close` / `cores` deste experimento e a versão `v4` da Tarefa 12, que são o mesmo código com a mesma afinidade em jobs diferentes, deram 1,009 / 0,940 / 0,586 s e 1,061 / 0,956 / 0,623 s com 4, 8 e 16 threads: diferença de 5% a 6%.

Conclusão

A escalabilidade deste código **muda mais com a afinidade do que mudou com as quatro otimizações da Tarefa 12** na faixa de 4 a 16 threads. Trocar uma variável de ambiente rendeu $2,4\times$ onde reescrever o laço rendeu bem menos.

O motivo é específico e vale enunciar: o programa é limitado por memória, e a política de afinidade decide quanto cache e quantos controladores de memória o programa alcança. `spread` maximiza os dois desde poucas threads; `close` só alcança o mesmo quando o número de threads já é grande o bastante para transbordar os CCX por conta própria.

Daí a regra prática que este experimento sustenta: para código limitado por memória e poucas threads, usar `spread`. Para código limitado por CPU que compartilha dados entre threads vizinhas, `close` tende a ser melhor, porque mantém as threads perto no cache. E `master` não serve para nada além de demonstrar o pior caso.

Vale registrar que o melhor tempo absoluto em 64 threads não veio do OpenMP, e sim do sistema operacional: `numactl --interleave=all` combinado com `close`. Em um programa cujo gargalo é colocação de páginas, a ferramenta do sistema resolveu melhor do que a política da biblioteca.

Código

afinidade.c · 337 linhas

```
1 #define _GNU_SOURCE
2
3 #include <dirent.h>
4 #include <malloc.h>
5 #include <math.h>
6 #include <omp.h>
7 #include <sched.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12
13 #define MAX_THREADS 512
14
15 typedef struct {
16     int n;
17     double h;
18     double viscosidade;
19     double dt;
20 } Malha;
21
22 static double agora(void)
23 {
24     struct timespec t;
25     clock_gettime(CLOCK_MONOTONIC, &t);
26     return (double)t.tv_sec + (double)t.tv_nsec * 1e-9;
27 }
28
29 static double *aloca(int n)
30 {
```

[illegible]

```

120                                     double *un, double *vn, int i, int j)
121 {
122     const int n = m->n;
123     const int i_ant = (i + n - 1) % n;
124     const int i_prox = (i + 1) % n;
125     const int j_ant = (j + n - 1) % n;
126     const int j_prox = (j + 1) % n;
127     const int centro = i * n + j;
128     const double fator = fator_de(m);
129
130     const double laplaciano_u = u[i_prox * n + j] + u[i_ant * n + j]
131                               + u[i * n + j_prox] + u[i * n + j_ant]
132                               - 4.0 * u[centro];
133     const double laplaciano_v = v[i_prox * n + j] + v[i_ant * n + j]
134                               + v[i * n + j_prox] + v[i * n + j_ant]
135                               - 4.0 * v[centro];
136
137     un[centro] = u[centro] + fator * laplaciano_u;
138     vn[centro] = v[centro] + fator * laplaciano_v;
139 }
140
141 static inline void linha_interior(const Malha *m, const double *u, const double *v,
142                                  double *un, double *vn, int i)
143 {
144     const int n = m->n;
145     const double fator = fator_de(m);
146
147     const double *u_ant = u + (i - 1) * n;
148     const double *u_cen = u + i * n;
149     const double *u_prox = u + (i + 1) * n;
150     const double *v_ant = v + (i - 1) * n;
151     const double *v_cen = v + i * n;
152     const double *v_prox = v + (i + 1) * n;
153     double *un_cen = un + i * n;
154     double *vn_cen = vn + i * n;
155
156     for (int j = 1; j < n - 1; j++) {
157         const double laplaciano_u = u_prox[j] + u_ant[j]
158                                   + u_cen[j + 1] + u_cen[j - 1]
159                                   - 4.0 * u_cen[j];
160         const double laplaciano_v = v_prox[j] + v_ant[j]
161                                   + v_cen[j + 1] + v_cen[j - 1]
162                                   - 4.0 * v_cen[j];
163         un_cen[j] = u_cen[j] + fator * laplaciano_u;
164         vn_cen[j] = v_cen[j] + fator * laplaciano_v;
165     }
166 }
167
168 static inline void linha_com_bordas(const Malha *m, const double *u, const double *v,
169                                     double *un, double *vn, int i)
170 {
171     const int n = m->n;
172
173     celula_periodica(m, u, v, un, vn, i, 0);
174     celula_periodica(m, u, v, un, vn, i, n - 1);
175
176     if (i > 0 && i < n - 1)
177         linha_interior(m, u, v, un, vn, i);
178     else
179         for (int j = 1; j < n - 1; j++)
180             celula_periodica(m, u, v, un, vn, i, j);
181 }
182
183 static void troca(double **a, double **b)
184 {
185     double *t = *a;
186     *a = *b;
187     *b = t;
188 }
189
190 static void inicializa(const Malha *m, double *u, double *v, double *un, double *vn)
191 {
192     const int n = m->n;
193     #pragma omp parallel for schedule(static)
194     for (int i = 0; i < n; i++)
195         for (int j = 0; j < n; j++) {
196             const int k = i * n + j;
197             u[k] = 1.0;
198             v[k] = 0.0;
199             un[k] = 0.0;
200             vn[k] = 0.0;
201         }
202     u[(n / 2) * n + n / 2] += 1.0;
203 }
204
205 static void simular(const Malha *m, double **pu, double **pv,
206                    double **pun, double **pvn, int passos)
207 {
208     double *u = *pu, *v = *pv, *un = *pun, *vn = *pvn;

```

```

209
210 #pragma omp parallel firstprivate(u, v, un, vn)
211 {
212     for (int t = 0; t < passos; t++) {
213 #pragma omp for schedule(static)
214         for (int i = 0; i < m->n; i++)
215             linha_com_bordas(m, u, v, un, vn, i);
216
217         troca(&u, &un);
218         troca(&v, &vn);
219     }
220 }
221
222 if (passos % 2 == 1) {
223     troca(pu, pun);
224     troca(pv, pvn);
225 }
226 }
227
228 /* ----- medicao */
229
230 static int comparar_double(const void *a, const void *b)
231 {
232     double x = *(const double *)a, y = *(const double *)b;
233     return (x > y) - (x < y);
234 }
235
236 static double mediana(double *v, int n)
237 {
238     qsort(v, (size_t)n, sizeof(double), comparar_double);
239     return v[n / 2];
240 }
241
242 static Malha malha_de(int n)
243 {
244     const double r = 0.20;
245     Malha m;
246     m.n = n;
247     m.h = 1.0 / (double)n;
248     m.viscosidade = 0.01;
249     m.dt = r * m.h * m.h / m.viscosidade;
250     return m;
251 }
252
253 static double medir(const Malha *m, int passos, int execucoes)
254 {
255     double *tempos = malloc((size_t)execucoes * sizeof(double));
256     double *u = aloca(m->n), *v = aloca(m->n);
257     double *un = aloca(m->n), *vn = aloca(m->n);
258
259     for (int e = 0; e < execucoes; e++) {
260         double *a = u, *b = v, *c = un, *d = vn;
261         inicializa(m, a, b, c, d);
262
263         double t0 = agora();
264         simular(m, &a, &b, &c, &d, passos);
265         tempos[e] = agora() - t0;
266     }
267
268     double t = mediana(tempos, execucoes);
269     free(tempos); free(u); free(v); free(un); free(vn);
270     return t;
271 }
272
273 static void imprimir_mapa(const char *rotulo, int threads)
274 {
275     int cpus[MAX_THREADS];
276     observar(threads, cpus);
277
278     printf("# mapa %s com %d threads\n# thread:cpu ", rotulo, threads);
279     for (int i = 0; i < threads && i < 32; i++)
280         printf("%d:%d ", i, cpus[i]);
281     if (threads > 32)
282         printf("...");
283     printf("\n");
284 }
285
286 int main(int argc, char *argv[])
287 {
288     if (argc < 3) {
289         fprintf(stderr, "uso: %s <rotulo> <n> [passos] [execucoes] [lista_threads]\n", argv[0]);
290         return 1;
291     }
292
293     mallopt(M_MMAP_THRESHOLD, 128 * 1024);
294
295     const char *rotulo = argv[1];
296     int n = atoi(argv[2]);
297     int passos = (argc > 3) ? atoi(argv[3]) : 200;

```

```

298     int execucoes = (argc > 4) ? atoi(argv[4]) : 5;
299     const char *lista = (argc > 5) ? argv[5] : "1,4,8,16,32,64,128";
300
301     Malha m = malha_de(n);
302
303     int contagens[64], total = 0;
304     for (const char *p = lista; *p && total < 64; ) {
305         contagens[total++] = atoi(p);
306         while (*p && *p != ',') p++;
307         if (*p == ',') p++;
308     }
309
310     const char *base = getenv("TEMPO_BASE");
311     double t1 = (base != NULL) ? atof(base) : 0.0;
312     int cpus[MAX_THREADS], l3[MAX_THREADS], numa[MAX_THREADS];
313
314     for (int k = 0; k < total; k++) {
315         int p = contagens[k];
316
317         observar(p, cpus);
318         for (int i = 0; i < p; i++) {
319             l3[i] = dominio_l3(cpus[i]);
320             numa[i] = no_numa(cpus[i]);
321         }
322
323         double t = medir(&m, passos, execucoes);
324         if (t1 == 0.0)
325             t1 = t;
326
327         printf("%5s,%d,%d,%.6f,%.4f,%.4f,%d,%d,%d\n",
328             rotulo, p, n, t, t1 / t, (t1 / t) / (double)p,
329             distintos(l3, p), distintos(numa, p), distintos(cpus, p));
330         fflush(stdout);
331     }
332
333     if (getenv("MAPA") != NULL)
334         imprimir_mapa(rotulo, 8);
335
336     return 0;
337 }

```